

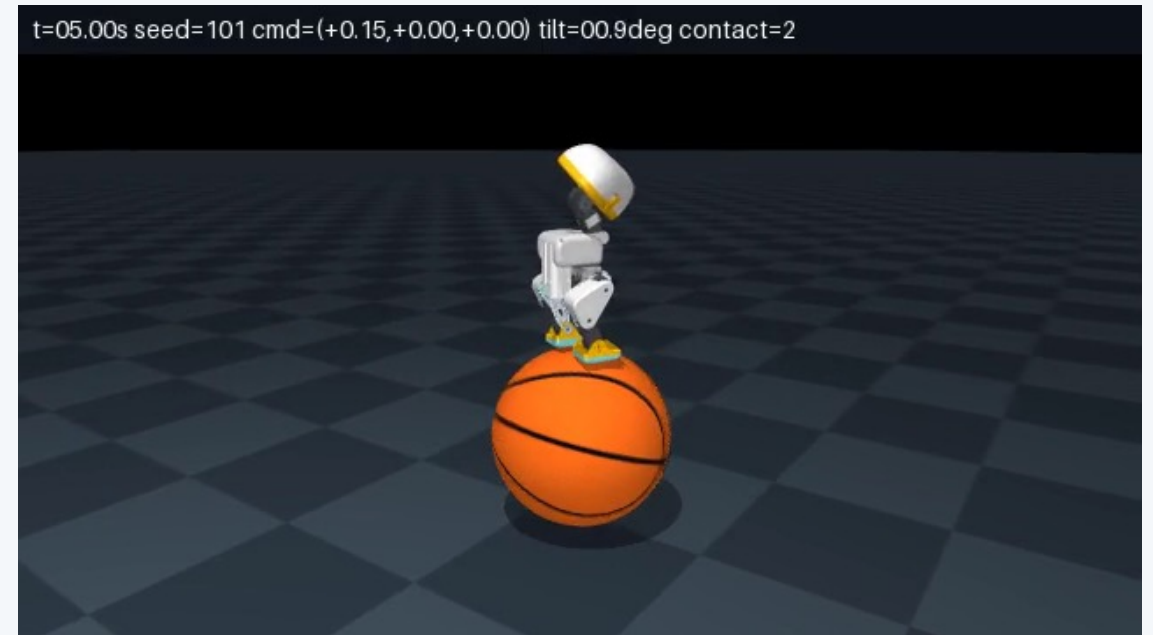
LAB 06

Basketball balance

Basketball: Blind Recurrent Balance, Not Steering

Environment design · PPO · Honest verification

BUILD → TRAIN → VERIFY → EXPLAIN



What you will build — and what the evidence says

- 01** Use case: Teach recurrent policy state, actor warm-starts, physical-assistance curricula, and honest task decomposition. The retained policy demonstrates unassisted balance on a moving free basketball, while commanded rolling and general steering remain unmastered.
- 02** By the end: describe the scene, map observations to actions, explain the reward, trace one PPO update, run a bounded experiment, and defend an evaluation claim.
- 03** The demonstrated result is balance, not stationary world-position hold, commanded rolling, or steering.
- 04** Scope: local MuJoCo simulation. Do not deploy these classroom artifacts to a physical robot; hardware transfer requires a separate validated process.

Materials and prerequisites · 1/3

- 01 Microduck full-collision MuJoCo model with CPU BAM actuators and 50 Hz policy control
- 02 A free basketball sphere with radius 0.12 m and mass 0.62 kg
- 03 microduck-playground/src/mjlab_microduck/robot/assets/basketball/basketball.obj and basketball.png for the non-colliding visual mesh and texture
- 04 microduck-playground/artifacts/basketball/checkpoint.pt and policy.onnx as the hash-verified b11 source actor

Materials and prerequisites · 2/3

- 05 `rlx/rlx/environments/basketball.py` for scene physics, assistance, reward, termination, and metrics
- 06 `rlx/rlx/models/basketball.py` for the LSTM256 actor, fresh critic, recurrent ONNX export, and parity test
- 07 `rlx/examples/ppo_microduck_basketball.py` for local recurrent PPO continuation
- 08 `rlx/scripts/eval_basketball_local.py` for deterministic first-fall evaluation and rendering

Materials and prerequisites · 3/3

09

rlx/runs/studio/basketball/basketball-balance-01/ for the retained Studio policy, evaluation, and preview

10

rlx/artifacts/basketball-local-20260910/ for the 102,400-step fixed-command training record

Environment and policy contract · 1/3

- 01** The physical sphere center starts at $z=0.121$ m. The robot root starts at $z=0.368$ m, upright near the ball apex; the task does not teach climbing onto the ball or floor recovery.
- 02** The physics step is 0.005 s with four substeps per action, giving a 0.02 s control interval and 50 Hz policy rate.
- 03** Observation contract: float32[61]. Indices 0:3 trunk angular velocity; 3:6 projected gravity; 6:20 joint offsets; 20:34 one-step-lagged joint velocity; 34:48 previous raw action; 48:51 velocity command; 51:61 zero padding.
- 04** Action contract: 14 joint-target offsets. Applied targets are default pose plus $\text{clip}(\text{action}, -4, 4)$; the observation preserves the previous raw action.

Environment and policy contract · 2/3

- 05 The actor receives no ball position, velocity, rotation, contact, radius, or world-position signal. It is a blind recurrent actor that can infer dynamics only through proprioceptive history.
- 06 Recurrent ONNX inputs are obs [1,61], h_in [1,1,256], and c_in [1,1,256]; outputs are actions [1,14], h_out [1,1,256], and c_out [1,1,256].
- 07 Hidden and cell state carry across ordinary steps and command changes, then reset on trial, episode, policy, environment, or recovery boundaries.
- 08 The physical assistance ladder is hold levels 1, 0.5, 0.25, 0.1, 0.03, and 0. It applies restoring force and torque through simulation only; rewards remain unchanged.

Environment and policy contract · 3/3

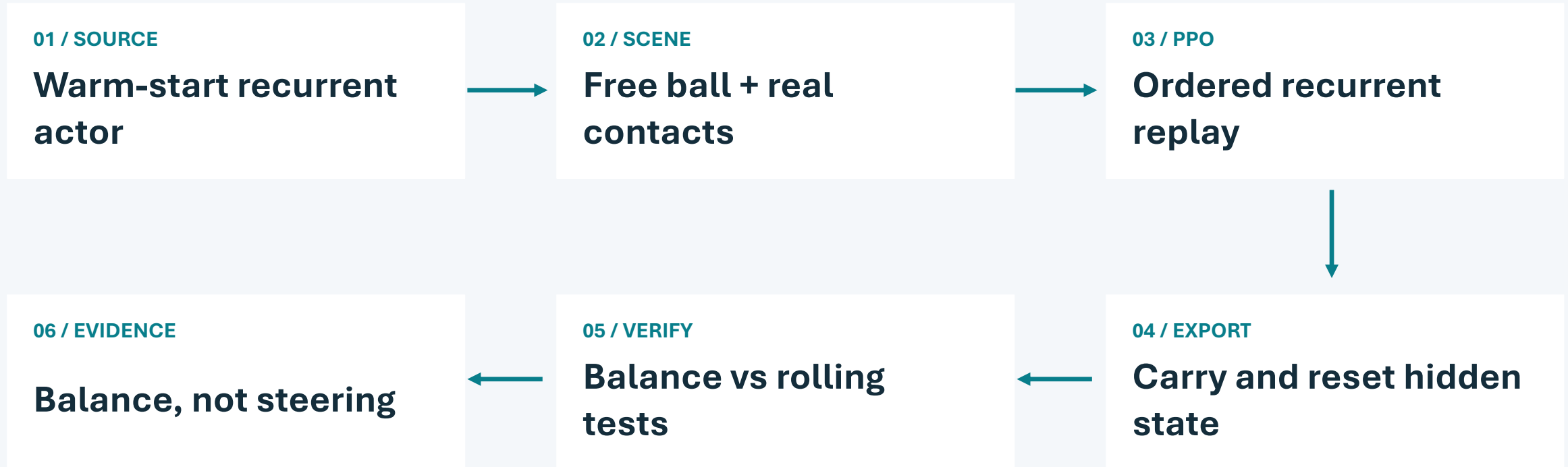
09

The retained fixed-command run trained with hold 0 and curriculum disabled. Evaluation always forces hold 0 and never auto-resets a failed trial.

10

The direct evaluation uses 60 s, 3,000 steps, seeds 101/202/303, randomized initial yaw, zero command and 0.15 m/s forward command, no pushes, no observation noise, and no mass/friction domain randomization.

How this case is built: one evidence loop



Failure is data: change one cause, retrain, and repeat the same evaluation gates.

Build the case step by step · 1/3

- 01 Verify the supplied b11 checkpoint and ONNX hashes before loading the source actor.
- 02 Build the free-ball MuJoCo scene with a colliding sphere and a separate massless, non-colliding visual mesh.
- 03 Reset the robot near the apex and zero recurrent hidden and cell state.
- 04 Transfer the mature recurrent actor and freeze its observation normalizer; initialize a fresh 61-input feedforward critic and fresh Adam optimizer.

Build the case step by step · 2/3

- 05 Collect complete time-by-environment recurrent rollouts, preserving the initial hidden/cell tensors and episode-start masks.
- 06 Replay the full ordered recurrent sequence during every PPO epoch; do not shuffle observations as independent rows.
- 07 Apply KL early stopping at target 0.02, save intermediate checkpoints, export recurrent ONNX, and test a state reset during parity comparison.
- 08 Evaluate zero-command balance and commanded-forward rolling as separate classifications.

Build the case step by step · 3/3

09

For each trial, carry recurrent state until first fall or exactly 3,000 steps; do not hide failure with automatic resets.

10

Check support, tilt, root offset, ball travel, ball rotation, directional progress, velocity tracking, assistance, and contact exploits.

11

Inspect both the zero-command balance video and the failed forward-command rolling video before writing the classroom claim.

PPO: improve the actor without trusting a huge step

$$\text{ratio} = \pi_{\text{new}}(a \mid o) / \pi_{\text{old}}(a \mid o)$$

Maximize $\text{mean}[\min(\text{ratio} \times A, \text{clip}(\text{ratio}, 1 - \epsilon, 1 + \epsilon) \times A)]$

Actor: chooses joint actions. Critic: estimates future discounted reward.

A = advantage. ϵ = clipping range. Old log probabilities stay fixed for each collected batch.

Follow one rollout through the learner

- 01 Collect observations, sampled actions, old log probabilities, values, rewards and episode-boundary flags using the current stochastic actor.
- 02 Compute TD residuals and generalized advantage estimates (GAE). A true terminal has zero continuation value; a timeout may bootstrap the saved final observation.
- 03 Cut advantage traces at reset boundaries. Never bootstrap a timeout from the first observation of the next episode.
- 04 Reuse this batch for PPO minibatch updates, then collect fresh on-policy data. Export the deterministic actor and its observation transform for evaluation.

Paper exercise: count samples and clip the objective

- 01 Illustrative arithmetic, NOT this run: $16 \text{ environments} \times 128 \text{ steps} = 2,048$ fresh transitions per rollout. Four epochs reuse those transitions; they do not create 8,192 new samples.
- 02 With $\epsilon = 0.2$, ratio = 1.4 and $A = +2$: $\min(2.8, 2.4) = 2.4$. The minimized policy term is -2.4 before averaging and additional loss terms.
- 03 With the same ratio and $A = -2$: $\min(-2.8, -2.4) = -2.8$. Explain why clipping is asymmetric once the sign of the advantage changes.
- 04 Run from repo root: `python3 docs/microduck-training-student-book/learning_lab.py --gae`. This is a deterministic arithmetic lab, not new policy training.

Reward design: what the agent is paid to do · 1/3

- 01 name: linear_tracking | weight: 1 | sign: reward | definition: $\exp(-\text{sum}((\text{body_linear_velocity_xy} - \text{command_xy})^2) / 0.1)$
- 02 name: yaw_tracking | weight: 0.5 | sign: reward | definition: $\exp(-(\text{gyro_z} - \text{command_yaw})^2 / 0.5)$
- 03 name: upright | weight: 1 | sign: reward | definition: $\exp(-\text{sum}(\text{projected_gravity_xy}^2) / 0.09)$
- 04 name: centered | weight: 2 | sign: reward | definition: $\exp(-\text{sum}(\text{horizontal trunk-to-ball offset}^2) / 0.0016)$

Reward design: what the agent is paid to do · 2/3

- 05 name: feet_on_ball | weight: 1 | sign: reward | definition: $\text{mean}(\exp(-\text{foot-to-ball surface gap squared} / 0.0004))$
- 06 name: height | weight: 1 | sign: reward | definition: $\exp(-(\text{relative trunk height} - \text{ball radius} - 0.125) \text{ squared} / 0.0009)$
- 07 name: ball_speed_penalty | weight: 0.05 | sign: penalty | definition: $-\min(\text{sum}(\text{ball horizontal velocity squared}), 100)$
- 08 name: angular_velocity_penalty | weight: 0.05 | sign: penalty | definition: $-\min(\text{sum}(\text{trunk roll/pitch angular velocity squared}), 100)$

Reward design: what the agent is paid to do · 3/3

09

name: action_rate_penalty | weight: 0.2 | sign: penalty | definition: -
min(sum((last_action - previous_action) squared), 100)

10

name: reward_scale | weight: 0.02 | sign: time scaling | definition: Every weighted
term is multiplied by CTRL_DT=0.02 before summation; the same weights are used at
every hold/curriculum level.

Recorded training recipe · 1/2

- 01 config: warm_start | value: b11 iteration 6999 actor, frozen normalizer, fresh local critic and fresh Adam optimizer; not an upstream optimizer resume
- 02 config: fixed_command_run | value: 102,400 local transitions from 100 updates x 8 environments x 128 steps, command [0.15,0,0], seed 42
- 03 config: optimizer | value: learning rate 2e-6, 5 recurrent replay epochs maximum, target KL 0.02, maximum gradient norm 1.0
- 04 config: ppo_objective | value: gamma 0.99, GAE lambda 0.95, clip coefficient 0.2, value coefficient 1.0, entropy coefficient 0.01, normalized advantages

Recorded training recipe · 2/2

- 05 config: exploration_and_environment | value: initial action standard deviation 0.03, BAM actuator, hold 0, curriculum off, pushes off, observation noise off, domain randomization off
- 06 config: checkpointing | value: intermediate artifacts at updates 25, 50, and 75; final update 100; actor parameter L2 change 0.042958568781614304
- 07 config: telemetry_fields | value: update; loss; policy_loss; value_loss; entropy; gradient_norm; approx_kl; epochs_completed; kl_early_stop; hold_min; hold_max; completed_episodes
- 08 config: evidence_gap | value: The Studio directory has no independent training trace. Charts use the archived source adaptation summary (100 PPO updates), whose checkpoint SHA-256 exactly matches the imported Studio checkpoint. No historical training reward series was retained.

Hands-on commands: inspect, smoke, train · 1/3

01

```
From .:  
rlx/.venv-microduck/bin/python -m pytest rlx/tests/test_basketball.py  
rlx/tests/test_basketball_policy.py rlx/tests/test_basketball_evaluation.py -q  
Run scene, recurrent policy, and evaluation-honesty contract tests.
```

02

```
From .:  
bash scripts/train-basketball.sh --output rlx/artifacts/basketball-smoke --num-envs 4 --  
updates 5 --steps 32 --learning-rate 2e-6  
Run the documented five-update training smoke in a fresh output directory.
```

Hands-on commands: inspect, smoke, train · 2/3

03

```
From .:  
bash scripts/train-basketball.sh --output rlx/artifacts/basketball-training --num-envs 8 --  
updates 100 --steps 128 --learning-rate 2e-6 --save-interval 25 --command .15 0 0  
Run the documented unassisted fixed-forward local adaptation.
```

04

```
From .:  
rlx/.venv-microduck/bin/python rlx/scripts/eval_basketball_local.py --policy  
rlx/artifacts/basketball-training/policy.onnx --output rlx/artifacts/basketball-evaluation  
--seconds 60 --seeds 101 202 303 --command 0 0 0 --command .15 0 0 --render  
Evaluate and render stationary balance and commanded rolling separately.
```

Hands-on commands: inspect, smoke, train • 3/3

05

From duck-viewer:

```
npm run dev
```

Open the Studio and inspect basketball-balance-01 with private recurrent state per duck.

Reward: does the logged signal match the task?

Recorded reward signals — different definitions, not interchangeable

mean_raw_return

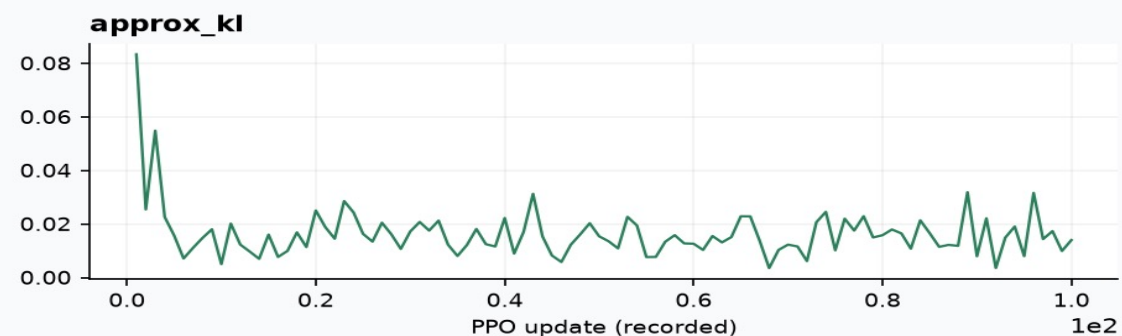
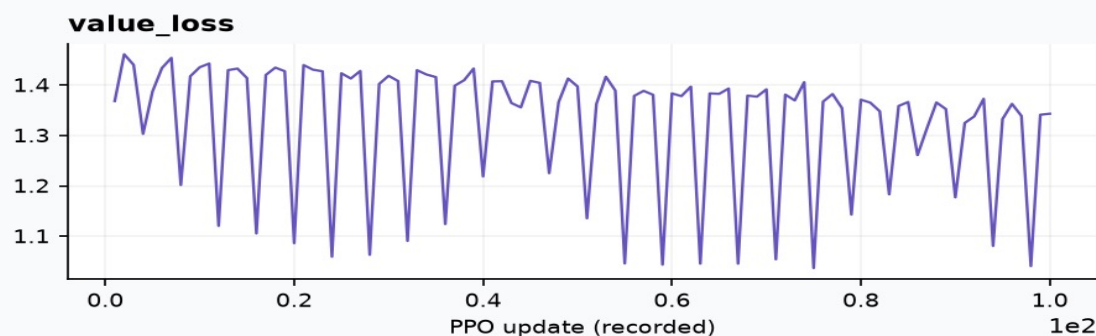
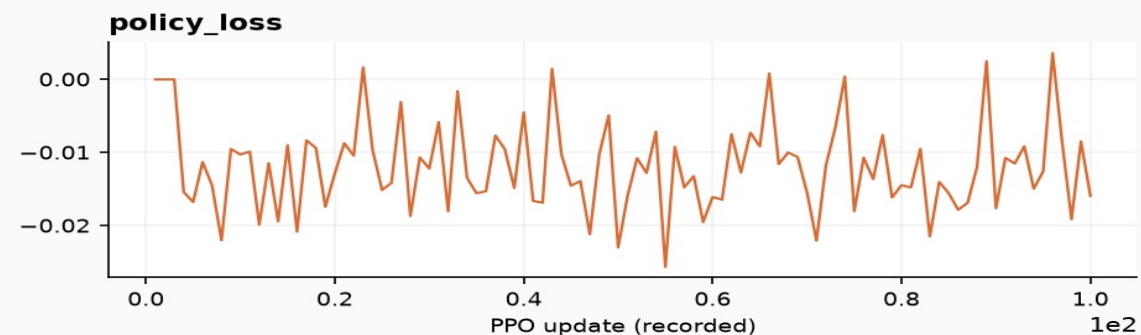
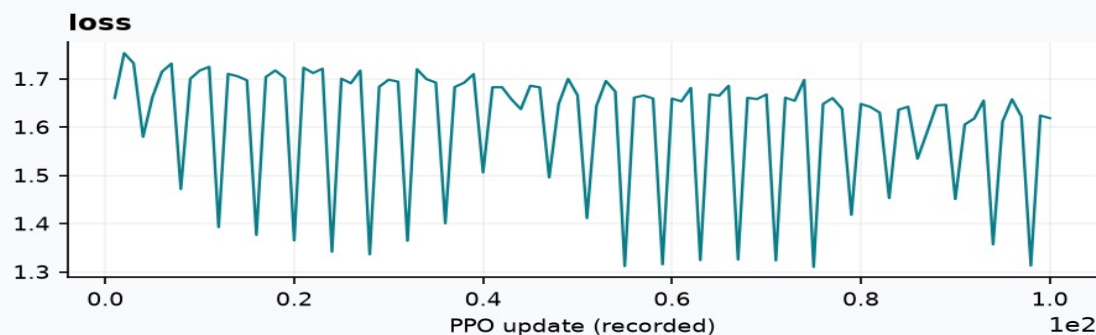
NOT RECORDED
No values invented

Not recorded: mean_raw_return, mean_reward, mean_episode_length, episode_count

Archived source adaptation; exact imported checkpoint hash match. No historical reward curve was retained.

Loss: inspect actor, critic and policy drift

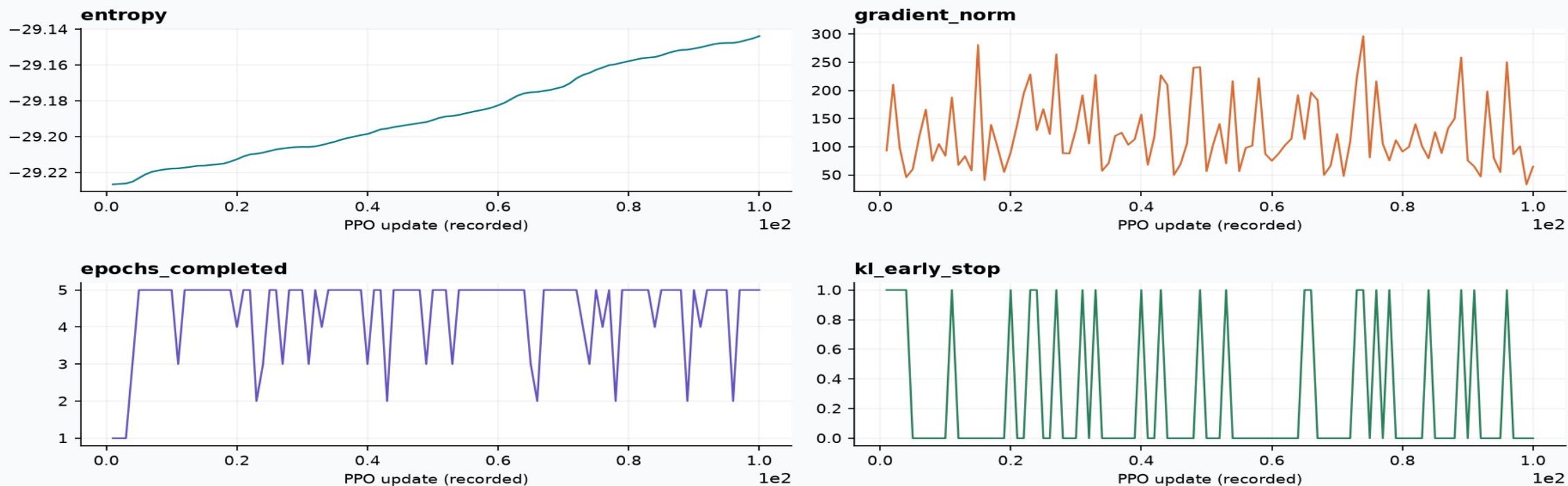
Recorded PPO optimization — raw values, no smoothing



Archived source adaptation; exact imported checkpoint hash match. No historical reward curve was retained.

Diagnostics: is the optimizer behaving?

Recorded update diagnostics — absent fields are explicit



Archived source adaptation; exact imported checkpoint hash match. No historical reward curve was retained.

Metric matrix: numerical audit of this trace · 1/2

- 01 approx_kl: first 0.08338 → final 0.01402; min 0.003539, max 0.08338; 100 logged samples.
- 02 entropy: first -29.23 → final -29.14; min -29.23, max -29.14; 100 logged samples.
- 03 kl_early_stop: first 1.000 → final 0.000; min 0.000, max 1.000; 100 logged samples.
- 04 loss: first 1.660 → final 1.619; min 1.310, max 1.753; 100 logged samples.

Metric matrix: numerical audit of this trace · 2/2

05

policy_loss: first $-7.637\text{e-}8$ → final -0.01592 ; min -0.02564 , max 0.003599 ; 100 logged samples.

06

value_loss: first 1.368 → final 1.343 ; min 1.037 , max 1.461 ; 100 logged samples.

Verification: define success before looking · 1/3

- 01 Verify source checkpoint and ONNX SHA256 before training; refuse arbitrary unverified source files.
- 02 Check that observation slots 51:61 remain zero and the actor receives no explicit ball state.
- 03 Confirm recurrent hidden/cell state carries through normal steps and resets exactly at episode boundaries.
- 04 Require recurrent PyTorch/ONNX parity over 40 steps with a state reset at step 20.

Verification: define success before looking · 2/3

- 05 Evaluate every trial for the full 3,000-step first-fall horizon with hold zero and no automatic resets.
- 06 Classify `sustained_balance` separately from `controlled_rolling` and `learned_success`.
- 07 Require no robot-floor contact, no non-foot ball support, adequate foot-ball contact, bounded tilt and offset, and zero assistance for balance credit.
- 08 For forward rolling, additionally require at least 3.15 m signed command progress, linear tracking MAE at most 0.09 m/s, yaw tracking MAE at most 0.5 rad/s, and sufficient physical ball rotation.

Verification: define success before looking · 3/3

09

Inspect the original MP4 rather than relying only on sparse contact-sheet frames or rotating texture.

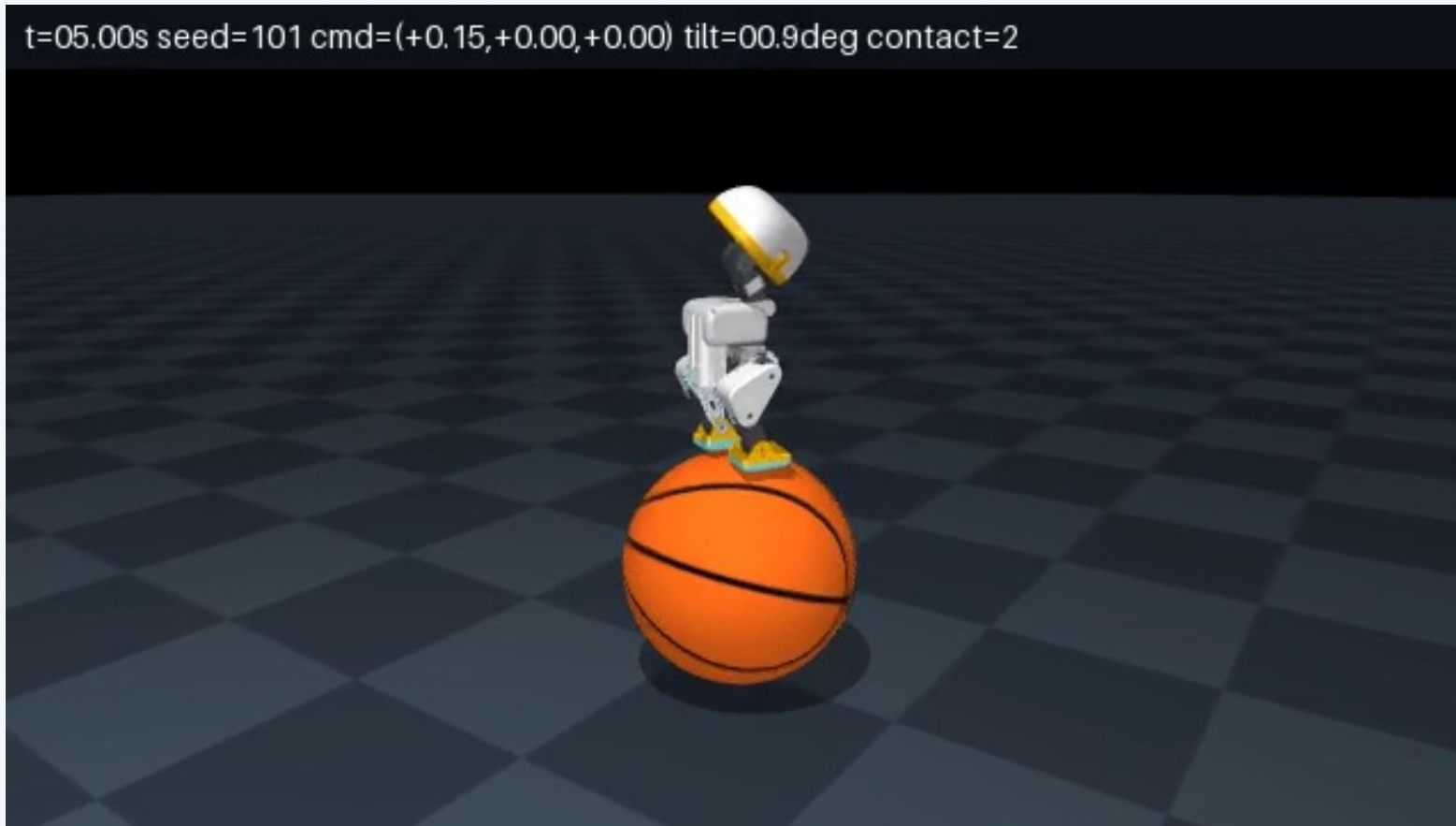
Results: what was actually measured • 1/2

- 01** fact: All 6 direct 60-second trials survived: seeds 101, 202, and 303 under zero command and 0.15 m/s forward command. | source: docs/basketball-showcase/evidence/evaluation.json
- 02** fact: All 6 trials met sustained-balance criteria, but 0 of 3 commanded-forward trials met controlled-rolling criteria; the aggregate report passed=false. | source: docs/basketball-showcase/evidence/evaluation.json
- 03** fact: Minimum foot-ball contact fraction was 0.9993333333333333, or 99.93333333333332 percent, and maximum tilt was 4.733669151434107 deg. | source: docs/basketball-showcase/evidence/evaluation.json
- 04** fact: The three forward-command signed progress values were 1.299956021320378 m, 0.3004973917973738 m, and 0.35659880459696325 m, each below the 3.15 m gate. | source: docs/basketball-showcase/evidence/evaluation.json

Results: what was actually measured · 2/2

- 05 fact: Mean forward tracking MAE across the three 0.15 m/s trials was 0.13764476597849953 m/s, above the 0.09 m/s gate. | source: docs/basketball-showcase/evidence/evaluation.json
- 06 fact: The fixed-command run added 102,400 transitions, changed actor parameters by L2 distance 0.042958568781614304, and exported with maximum recurrent parity error 1.430511474609375e-06. | source: rlx/artifacts/basketball-local-20260910/summary.json
- 07 fact: The final update recorded loss 1.6185429096221924, policy_loss -0.015916798263788223, value_loss 1.343021035194397, entropy -29.143863677978516, approximate_kl 0.014023307710886002, and epochs_completed 5. | source: rlx/artifacts/basketball-local-20260910/summary.json
- 08 fact: The retained Studio adapter separately evaluates seeds 101-103 and still reports 6 of 6 sustained balance with 0 of 3 commanded rolling passes. | source: rlx/runs/studio/basketball/basketball-balance-01/evaluation.json

Watch the actual saved rollout



BEFORE PLAY

Predict one success signal and one failure signal.

AFTER PLAY

Which claims does this clip support? What remains untested?

60.0 s saved MP4 • Click in PowerPoint Slide Show • Backup: videos/basketball-rollout.mp4

Limits and failure analysis · 1/3

- 01 The demonstrated result is balance, not stationary world-position hold, commanded rolling, or steering.
- 02 Lateral and yaw steering are not evaluated by the current Studio adapter, so full-skill success fails closed.
- 03 The robot starts on the ball; mounting, recovery from the floor, and transfer between surfaces are outside this task.
- 04 The actor is a warm-started mature b11 policy, not a policy trained from scratch by the local 102,400-step run.

Limits and failure analysis · 2/3

- 05 The local critic and optimizer are fresh, so this is not exact continuation of the upstream GPU training state.
- 06 Loss charts are archived SOURCE-ADAPTATION diagnostics, not a new Studio training run. The imported checkpoint hash matches; the ONNX file was re-exported. Historical training reward remains unavailable.
- 07 The six nominal local trials omit sensor noise, mass/friction randomization, and pushes; they do not reproduce the upstream stress battery.
- 08 Historical direct evaluation hashes predate the current source snapshot; the report is retained evidence, not a fresh byte-identical rerun.

Limits and failure analysis · 3/3

09

No hardware balance or safety claim is made.

Student lab: change one variable, keep the gates · 1/3

- 01 Trace h_{in}/c_{in} to h_{out}/c_{out} for five steps, insert an episode reset, and mark where both recurrent tensors must become zero.
- 02 Calculate the 102,400 transition count from environments, rollout steps, and updates.
- 03 Compute each weighted reward contribution for a supplied one-step state, including the final 0.02 control-time scale.
- 04 Compare `sustained_balance` and `controlled_rolling` criteria and identify the extra measurements needed for steering credit.

Student lab: change one variable, keep the gates · 2/3

- 05 Plot the three forward progress values against the 3.15 m threshold and explain why visible ball motion is insufficient.
- 06 Inspect the zero-command and forward-command videos and annotate support changes, drift, and camera-relative motion.
- 07 Read the final PPO loss record and explain why its finite values do not resolve the missing historical reward curve.
- 08 Propose a rolling-state initialization curriculum that changes exploration physics or spawns while leaving evaluation and reward weights fixed.

Student lab: change one variable, keep the gates · 3/3

09

Write separate evidence statements for source b11 performance, local fixed-command adaptation, and Studio playback.

Submit an experiment, not just a video

- 01 Before training: write one hypothesis, the changed parameter, the fixed baseline, seed list, acceptance criteria and time/transition budget.
- 02 During training: save the exact command, environment versions, initialization, checkpoint and raw reward/loss/KL metrics. Never overwrite the saved reference run.
- 03 After training: evaluate the frozen exported policy on matched seeds; include all failures, aggregate metrics, the full rollout video and a source-bound result table.
- 04 Rubric / 10: reproducibility 2; environment and reward explanation 2; PPO and metrics interpretation 2; honest verification 2; one evidence-based next experiment 2.

Source map and further study · 1/2

01

microduck_local/AGENTS.md

02

docs/student-book/microduck-lab-student-book.md

03

docs/basketball-bridge-book/chapters/en/03-ppo.md

04

docs/basketball-bridge-book/chapters/en/04-basketball.md

Source map and further study · 2/2

05

`docs/microduck-training-student-book/chapters/02a-ppo-code-lab.md`

06

`docs/microduck-training-student-book/chapters/02b-gae-workshop.md`

07

`scripts/scenarios/README.md`

08

`README.md`